

Projet : Système Q&A Médical RAG Multimodal

Analyse du document OMS (COVID-19)

Izadine Massar

18 décembre 2025

- 1 Objectifs du Projet
- 2 Architecture Technique
- 3 Mise en oeuvre et Code
- 4 Résultats et Conclusion

- **Problématique** : Extraire des informations fiables d'un document médical dense (OMS, 182 pages) sans entraînement lourd.
- **Solution** : Architecture **RAG (Retrieval-Augmented Generation)**.
- **Contraintes techniques** :
 - Exécution 100% sur CPU.
 - Approche Multimodale (Analyse des pages comme images).
 - Sécurité (Audit des réponses pour éviter les hallucinations).

Architecture du Système (Workflow)

Le système suit un pipeline en 4 étapes majeures :

- ➊ **Ingestion** : Conversion du PDF en images haute résolution.
- ➋ **Indexation** : Encodage des pages avec le modèle multimodal **CLIP**.
- ➌ **Retrieval** : Traduction de la requête et recherche de similarité cosinus.
- ➍ **Génération** : Synthèse en français par le LLM **TinyLlama**.

Étape 1 & 2 : Ingestion et Indexation

Nous transformons le PDF en images pour capturer le texte et les éléments visuels, puis nous utilisons CLIP pour créer des vecteurs.

```
1 # Extraction et indexation multimodale
2 model = SentenceTransformer('clip-ViT-B-32')
3 page_embeddings = model.encode(
4     [Image.open(p) for p in page_paths],
5     convert_to_tensor=True
6 )
7 # Resultat : Chaque page est un vecteur de dimension 512.
```

Succès : 182 pages indexées avec succès en batches.

Étape 3 : Recherche et Audit de Sécurité

L'audit est crucial pour rejeter les questions hors-sujet.

```
1 def security_audit(results, threshold=0.20):  
2     best_score = results[0]['score']  
3     if best_score < threshold:  
4         return False, "Alerte : Document non pertinent."  
5     return True, "Audit réussi."
```

Résultat de recherche

Question : "Traitements recommandés ?"

→ Page 0 (Score : 0.32) | Page 43 (Score : 0.26)

Statut : Validé par l'audit.

Étape 4 : Génération Multilingue

Le modèle lit l'anglais et génère une réponse structurée en français.

```
1 # Utilisation de TinyLlama pour la synthese
2 generator = pipeline("text-generation", model="TinyLlama-1.1B-Chat")
3 prompt = f"Use context to answer in FRENCH: {context}"
4 reponse = generator(prompt, max_new_tokens=150)
```

Performance : Le modèle fonctionne sur CPU avec une latence acceptable grâce à la quantification.

Requête Utilisateur (Français)

"Quels sont les traitements recommandés ?"

- **Traduction interne** : "What are the recommended treatments ?"
- **Réponse générée** : "Les traitements recommandés sont : Antiviraux, Anticoagulants, et Oxygénothérapie (HFNO)..."

Test de Sécurité

Question : "Recette de cuisine ?"

→ **Réponse du système** : "Alerte Sécurité : Score trop faible."

- **Réussite** : Système fonctionnel capable de traiter des documents complexes sur matériel modeste.
- **Points forts** : Multimodalité (CLIP) et barrière de sécurité intégrée.
- **Limites** : Capacité de contexte limitée à 2048 tokens pour TinyLlama.
- **Futur** : Intégration de modèles plus larges (Llama-3 8B) via GGUF pour plus de précision.